Pipelined RISC-V Simulator Supporting Hazard Detection and Data Forwarding

Members

Christian Watts

Alondra Conchas-Sanchez

Lab SUMMARY

In this lab, we extended our unpipelined RISC-V simulator from Lab 1 to a fully pipelined version capable of handling ALU and load/store instructions. Our pipeline design implements the standard 5-stage RISC-V pipeline:

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execution (EX)
4. Memory Access (MEM)
5. Writeback (WB)

We leveraged pipeline registers between each stage to hold intermediate values and control signals, ensuring data flows correctly between stages across cycles. Additionally, we implemented the print_program() function to output the instructions loaded into memory in assembly format, providing better visibility and debugging capability for our simulation.

1. Work Distribution

Christian Watts –

a) Implemented the MEM and WB stage functions
b) Implemented the print_program() function
c) Debugged and tested unified source code

Alondra Conchas-Sanchez –

a) Implemented the IF, ID, and EX stage functions
b) Created the decode() utility function
c) Debugged and tested unified source code

2. Milestones

1. Implement Instruction Fetch (IF) and Instruction Decode (ID) stages
2. Implement Execution (EX) stage and confirm correct ALU operations
3. Implement Memory Access (MEM) and Writeback (WB) stages
4. Implement print_program() function
5. Final integration, debugging, and testing
6. Final report generation

3. Obstacles

- Asdf

4. Detailed Implementation

1. Instruction Fetch

   a) Fetches the instruction from memory using the program counter

   b) Stores the instruction and next PC into the IF_ID pipeline register

   c) Increments the PC by 4, preparing for the next fetch

```
/* implement the pipeline stage */
uint32_t IR = mem_read_32(CURRENT_STATE.PC);
uint32_t next_pc = CURRENT_STATE.PC + 4;

/* implement the IF_ID Pipe_Reg */
IF_ID.IR = IR;
IF_ID.PC = next_pc;
NEXT_STATE.PC = next_pc;
```

2. Instruction Decode

a) Decodes instruction into its opcode, registers, and immediate values using the decode() function
b) Reads values from the register file into temporary pipeline registers
c) These values will be used in the EX stage

```
decode(&opcode, &f3, &f7, &rd, &rs1, &rs2, &imm);
uint32_t regA = 0;
uint32_t regB = 0;
uint32_t rs = 0;
uint32_t rt = 0;
uint32_t IMM = 0;

switch(opcode){
    case R_OPCODE:
        rs = rs1;
        rt = rs2;
        regA = CURRENT_STATE.REGS[rs];
        regB = CURRENT_STATE.REGS[rt];
        break;

    case IMM_ALU_OPCODE:
    case LOAD_OPCODE:
        rs = rs1;
        regA = CURRENT_STATE.REGS[rs];
        if (imm & 0x800){
            imm |= BIT_MASK_20;
            IMM = imm;
        }
```

3. Execution

a) Performs ALU operations based on instruction type

i) Memory instructions (load/store): calculate effective address

ii) Register operations: perform arithmetic/logic

iii) Register-immediate operations: compute immediate-based ALU result

b) Stores the result in EX_MEM.ALUOutput

```
    uint32_t alu_output = 0;
    uint8_t opcode = GET_OPCODE(ID_EX.IR);
    uint8_t f3 = GET_FUNCT3(ID_EX.IR);
    uint8_t f7 =  ID_EX.IR >> 25 & BIT_MASK_7;

    switch(opcode){
/* Memory Reference (load/store)
    case LOAD_OPCODE:
    case STORE_OPCODE:
        alu_output = ID_EX.A + ID_EX.imm;
        break;

/* Register-Register Operation
    case R_OPCODE:
        switch (f3){
            case 0x00:
                if (f7 == 0){
                    alu_output = ID_EX.A + ID_EX.B;
                }
                else{
                    alu_output = ID_EX.A - ID_EX.B;
                }
            break;

            case 0x4:
                alu_output = ID_EX.A ^ ID_EX.B;
            break;
```

4. Memory Access

a) If instruction is load, reads data from memory

b) If instruction is store, writes data to memory

```
uint32_t opcode = GET_OPCODE(EX_MEM.IR);

MEM_WB.IR = EX_MEM.IR;
MEM_WB.ALUOutput = EX_MEM.ALUOutput;

if (opcode == LOAD_OPCODE) {
    MEM_WB.LMD = mem_read_32(EX_MEM.ALUOutput);
} else if (opcode == STORE_OPCODE) {
    mem_write_32(EX_MEM.ALUOutput, EX_MEM.B);
}
```

5. Writeback

a) Writes the result back to the register file

b) Increments INSTRUCTION_COUNT for completed instructions

```
uint32_t opcode = GET_OPCODE(MEM_WB.IR);
uint8_t rd = (MEM_WB.IR >> 7) & BIT_MASK_5;
uint8_t rt = rd;  // For I-type load

if (opcode == R_OPCODE) {
    NEXT_STATE.REGS[rd] = MEM_WB.ALUOutput;
} else if (opcode == IMM_ALU_OPCODE) {
    NEXT_STATE.REGS[rt] = MEM_WB.ALUOutput;
} else if (opcode == LOAD_OPCODE) {
    NEXT_STATE.REGS[rt] = MEM_WB.LMD;
}

if (opcode == R_OPCODE || opcode == IMM_ALU_OPCODE || opcode == LOAD_OPCODE) {
    INSTRUCTION_COUNT++;
}
```